AI ASSURANCE LAB PLATFORM · CISCO × ANTHROPIC · FULL-DAY HANDS-ON WORKSHOP

AI Assurance — Partner MCP Day

Lab Guide — turning plain English into real network action with Claude & MCP

ThousandEyes | Meraki | Splunk

Print / Save PDF

START HERE

Before you start

Five minutes of context so every lab makes sense.

What is MCP, in one paragraph

The **Model Context Protocol (MCP)** is an open standard — think of it as a *USB-C port for AI*. It lets Claude plug into external tools and data (like ThousandEyes or Meraki) through a standard interface. You ask Claude a question in plain English; Claude decides which tools to call, calls them, and weaves the results into its answer. You never touch an API or write a script.

Everything runs in one place — this web app

- **No installs, no config files, no Claude Desktop.** You're already in the lab platform. The chat on the right talks to Claude on AWS Bedrock, which calls ThousandEyes and Meraki through their official hosted MCP servers on your behalf.
- **One-time setup per service:** open ⚙ **Credentials** in the header, paste in your ThousandEyes OAuth Bearer token and Meraki API token, and click **Test Connection**. That's it — no OAuth pop-ups to click through, no local Python servers, no JSON config to edit.
- Once a service shows **Connected ✓**, just ask Claude questions in the chat — it automatically knows which tools it has available and uses them for you.
- **Curious what Claude actually did behind the scenes?** Every answer that involved tool calls shows small collapsed rows above the text — `🔧 tool name` for each tool it called (click to expand and see exactly

what was sent and what came back), and 💭 for any reasoning it wrote along the way. They're collapsed by default so they stay out of your way, but expanding them is a great way to see which MCP tools solved a given question.

WHAT'S IN THIS GUIDE

- **1 · THOUSANDEYES** — connect via Credentials & 4 labs — Required
- **2 · MERAKI** — connect via Credentials & lab prompts — Required
- **3 · SPLUNK** — AI Assistant (3 labs) + MCP token & 2 investigation exercises, ~50–65 min (bonus) — Optional

Modules 1 & 2 (and connecting Module 3) all run inside this web app — nothing to install. Module 3's labs need a running Splunk MCP server (yours or the facilitator's) as bonus content. Use the **PRINT / SAVE PDF** button at the top of the page to save a copy of this guide for your event.

MODULE 1 · REQUIRED

↑ back to top

ThousandEyes MCP

Connect this web app to Cisco's official ThousandEyes MCP server — one OAuth Bearer token, pasted once, no install.

🎟️ **No ThousandEyes account?** See the lab proctor — demo licenses and an OAuth Bearer token are provided for today.

Connect ThousandEyes (2 minutes, in this app)

The ThousandEyes MCP server is cloud-hosted by Cisco. This lab platform talks to it on your behalf using your own personal OAuth Bearer token — nothing to install, nothing to configure locally.

1. **Get your ThousandEyes OAuth Bearer token**
   In the ThousandEyes UI, go to **Account Settings → Users and Roles → User API Tokens** (or ask your proctor for the demo account's token). Copy the **OAuth Bearer Token** — you'll paste it once.

2. **Open Credentials in this app**

Click ⚙ **Credentials** in the header of the chat page.

3 **Paste your token and save**

Paste the token into the **ThousandEyes** field (labeled **OAuth Bearer Token**) and click **Save Token**.

4 **Test the connection**

Click **Test Connection**. You should see **Connected ✓** — that's it, you're done. The header status dot next to "ThousandEyes" will also turn green.

Verify it's connected

```
What ThousandEyes MCP tools are available to you? List them all, and group them into categories based on their function.
```

You should see tools like `list_alerts`, `get_anomalies`, `create_synthetic_test`, `search_outages` and many more, organized into categories (e.g. Tests, Alerts, Agents, BGP/Path Visualization, Dashboards, Endpoint Monitoring). If Claude says it has no tools, go back to **Credentials** and re-check the token and Test Connection result.

ℹ Your token is encrypted before it's stored and is never shown back to you or exposed to the browser in plaintext. Your sign-in must have the ThousandEyes **API Access** permission (the demo accounts already do).

Find and select your Account Group

```
List all ThousandEyes account groups you have access to.
```

🏷 **Know which Account Group you're in.** Every student gets their own dedicated ThousandEyes Account Group inside the shared organization **"AI-Powered Network Observability Day"**, named after you — `Your Name (your.email@example.com)`. All the tests, alerts, and data you see and create in the labs live in *that* group only, not your classmates'. In the list Claude returns, find the account group with *your own* name and email, note its **AID** (account group ID), then tell Claude to use that AID, for example:

```
Use the account group with AID [your AID] for everything from here on.
```

If you ever see someone else's name there, double-check you picked the right AID before running any lab prompts below — tests created against the wrong account group won't show up as yours.

ThousandEyes Labs

📖 **One customer, one story.** The 5 labs below are chapters, not one-offs — Lab 1 picks a real customer and creates 5 `customer-demo`-tagged tests for them; every chapter after that (triage, SLA proof, anomaly detection, final report) works off those exact same tests. Every chapter happens entirely in this chat — you never need to open the ThousandEyes UI to tell this story.

⏱️ **Short on time?** Labs 1-2 are marked CORE — do these first, they're the ones everyone should get to. Labs 3-5 are STRETCH goals — great if you have time, but nothing later in this module or any other module depends on them, so don't worry if class ends before you get there.

LAB 1 ★ **Prospect Demo — from a domain to a customer-ready report** CORE

The flagship sales-engineering workflow. ~15–20 minutes, start to finish.

Step 1 — Discover the prospect's internet services

Open `servicefinder.thousandeyesresearch.com`, search the primary domain of a **real customer of yours** (use an actual account you work with — this makes the demo you build genuinely useful), and take a screenshot of the full "Results for domain" list. Claude will do the hostname extraction and selection for you in the next step — no need to pre-pick URLs yourself.

Step 2 — Ask Claude to create the tests

In Claude, attach your screenshot, then send:

```
Attached is a screenshot of a "Results for domain" list. Each tile is
a
hostname with its hosting provider underneath. Extract every tile
using only
what's visible. Flag any hostname that looks truncated or misread.

GOAL: pick the 4–5 best ThousandEyes tests to show this customer, to
get
them interested in the product. Then create them with the ThousandEyes
MCP
server. No questions; be conservative if anything is ambiguous.

1. CLASSIFY each hostname as Customer-Facing, Employee Productivity
(SaaS,
   collaboration, CRM, internal tools), or Other. Use the name and
provider
   as clues. Third-party SaaS such as Salesforce and Zoom is in scope.
```

```
   Exclude dev/beta/qa/sandbox hosts and Target-owned private hosts
   (e.g., guestwifi.ewips).

2. SELECT 5 hosts, favoring:
   - A mix of both categories (at least 2 of each)
   - A mix of providers (Fastly, Cloudflare, Salesforce, etc.), since
showing
     how many third parties sit in their delivery path is the story
   - Business-critical names (storefront, loyalty, support, CRM,
meetings)
   Show me the 5 picks with a one-line reason each, then continue.

3. BASELINE each pick with create_and_run_instant_test (HTTP Server,
   https://<host>) from one US Cloud Agent. If a pick fails to connect
or
   times out, replace it with the next-best candidate.

4. CREATE with create_synthetic_test: one HTTP Server test per pick,
   5-minute interval, 4 geographically diverse US Cloud Agents
   (list_cloud_enterprise_agents), named "DEMO | <host> | <category>".
   If the baseline returned a redirect or login page (3xx/401/403),
set the
   expected status to match so it doesn't alert as an outage. Skip any
host
   that already has a test (list_tests).

5. TAG all tests with key "demo" / value "customer-demo" (create_tag
if
   needed — ThousandEyes tags are key:value pairs, and creating one
requires
   setting access_type to "all"). Then assign that tag to all 5 tests.

6. OUTPUT a table: test name, ID, target, provider, category, baseline
   status, and why it makes a good demo story. Then list the hosts you
   skipped and why.
```

Step 3 — Pull the results (after ~5 minutes)

```
The tests tagged "customer-demo" should have their first results now.
Retrieve the latest metrics for every test with that tag. For each
show: page load time, HTTP response code, pass/fail, any errors, and
which agent ran it. Summarize as a table.
```

Step 4 — Generate the customer report ★

```
Using those results, write a professional customer-facing performance
report for [PROSPECT] with: 1) Executive summary, 2) Test coverage
table, 3) Key findings, 4) What this means for their business, 5) How
ThousandEyes solves it, 6) Recommended next steps (a 30-day trial).
Tone: trusted advisor, reference real numbers, not too technical.
Audience: a Network Manager / IT Director.
```

PUSH IT FURTHER

```
Turn this report into an email I can send to the IT Director — under
200 words, with a subject line referencing the specific issue you
found.
```

```
Based on what we found, which additional ThousandEyes test types would
you recommend, and what business question does each answer for this
prospect?
```

LAB 2 The Outage Call

CORE

Day 1 of the trial — your customer's NOC wants proof ThousandEyes would catch something the moment it happens. Prove the alerting and triage story in plain English, using both your own org and the live internet.

```
Check my own ThousandEyes org for alerts in the last 7 days (both
triggered and cleared). If there's anything there, summarize it. If
there's nothing yet, that's the expected, correct answer this early in
a trial with only a few minutes of test history — say so explicitly
rather than treating it as a dead end.
```

```
Now prove the platform is watching the whole internet, not just our
handful of tests: search for real outages in the last 24 hours. Pick
the most significant one and walk me through it like you'd walk a
customer through it — what happened, which provider or network, how
widespread, and what evidence proves it.
```

```
For that outage, what would a probable fault domain and
troubleshooting path look like if this were hitting my customer's
traffic instead? Use the supporting evidence from the data to make the
case.
```

```
Now check on my customer's own tests specifically: look at every test
tagged "customer-demo" and tell me if any of them are currently
showing errors, failures, or response times noticeably above their
baseline. This is what I'd tell the customer if they asked "is our
stuff okay right now?"
```

LAB 3 Proving the SLA

STRETCH

Your customer's champion needs to go to their boss with proof the trial is working — starting from day one, even with just a few minutes of data.

```
Find the test IDs for every test tagged "demo"="customer-demo" (look
them up directly rather than filtering metrics by tag, since tag-based
metric filters can silently return nothing). Then pull results for
those test IDs since they were created. For each show: uptime %,
average time-to-first-byte, and average response time. Rank the 5
hosts best to worst.

Be honest about the data window: if it's only been a few minutes, say
so explicitly and frame this as "Day 1 baseline" rather than a long-
term trend — that's a real and useful thing to show a customer on day
one of a trial.
```

```
Call out anything that dropped below a 99.5% uptime SLA or had a
noticeably slow TTFB compared to the rest. If nothing did, say so
clearly — don't manufacture a problem.
```

```
Write a short, 3-4 sentence "proof point" paragraph I could paste into
an email to the customer's IT Director, citing the specific numbers
above and framing it as "here's what we're already seeing on day one —
here's what a full week will show."
```

LAB 4 Catching It Before They Do

STRETCH

The real sales pitch isn't "it works" — it's "we'll know before you do." Prove it on the customer's own tests.

```
Find the test IDs for every test tagged "demo"="customer-demo" (look
them up directly rather than filtering by tag, since tag-based filters
can silently return nothing), then analyze those tests for metric
```

```
anomalies in the time since they were created. Flag anything
statistically unusual, even if it hasn't breached an alert threshold.

Note: anomaly detection needs an established baseline, so on tests
this new a clean "no anomalies found" is the expected, correct answer
— report it as a finding (clean baseline established) rather than
treating it as a dead end.
```

```
No anomalies is itself worth showing the customer — prove it with a
picture, not just a clean bill of health. Pick one of the "customer-
demo" tests and show me the path visualization: every network hop from
our agent to their host, with latency and loss at each hop. Call out
anything unusual (extra hops, a high-latency jump, an ASN change); if
it's clean end-to-end, say that explicitly — this is literally what
their traffic path looks like right now.
```

```
Are any of the "customer-demo" tests showing no data in the last 30
minutes? List them and suggest why they might be silent.
```

```
Pick one host from the "customer-demo" tests and run a fresh agent-to-
server instant test against it from a different Cloud Agent than the
one already testing it. This is the "watch me check it live, right
now" moment for the customer — results can take up to a minute to
populate, so check again if the first read comes back empty.
```

LAB 5 ★ **The Story** STRETCH

Bring it all together. One narrative, built entirely from this chat — no dashboard, no slides software, no manual copy-paste between tools.

```
Pull together everything from this session on my customer's "customer-
demo" tests: the outage/triage check from earlier, the SLA proof
numbers, and the anomaly/live-check result. Write one cohesive
executive narrative (under 400 words) that tells the story of this
trial's Day 1: what we watched, what we proved, what we caught early,
and why that matters for their business — plus one line on what a full
week of data will add. Tone: confident, data-backed, written for a
Network Manager / IT Director deciding whether to expand the trial.
```

PUSH IT FURTHER

```
Turn that narrative into 5 slide talking points I could present live,
each with a one-line "so what" for the business. Deliver it as an
```

```
interactive HTML slide deck I can click through right here in the chat
— one slide per screen with next/back controls, not plain text.
```

```
What's the one number or finding from this whole session that would
make the strongest case for expanding this trial? Make the case in two
sentences.
```

🏆 Notice what you just did: discovered the customer's services, created real tests, triaged an alert, proved an SLA, caught an anomaly, and wrote an executive narrative — all without opening `thousandeyes.com` once.

MODULE 2 · REQUIRED

↑ back to top

Meraki MCP

Cisco's official Meraki MCP server, already connected for you — no key, no setup, nothing to install.

✅ Nothing to set up — we've already connected a shared Meraki organization for everyone in this lab. No API key, no Organization ID, no Credentials page needed. If you'd rather use your own Meraki organization instead, you can add your own API key and Org ID on the **Credentials** page and it will automatically take priority.

🔒 The shared key is **read-only** — on purpose. You have full visibility into every network, device, client, and alert, but no prompt in these labs can change a config, so there's nothing to accidentally break. If Claude ever says it can't make a change, that's the read-only key doing its job, not a tool failure.

Verify it's connected

```
What Meraki MCP tools are available to you? List them all.
```

If Claude says it has no Meraki tools, let your proctor know — that would mean something's off with the shared connection, not something you need to fix.

Meraki Labs

📖 **One customer, two sites.** This customer has multiple branches in the same Meraki org. Lab 2 investigates a real historical wireless incident at their `BR03-Munich` branch using actual connection and latency history (not simulated data) — that's the one site in this org with real incident history to dig into. Every other lab works on their `HUB11-Amsterdam` hub: a healthy, well-configured site, which is exactly the right place to practice security review, performance proof, and clean documentation. The capstone (Lab 6) weaves both into one story. All of it happens in this chat — you never need to open the Meraki dashboard to tell it.

🎯 The prompts below are already filled in with real network and device names from this org. You're not limited to them, though — this is a real multi-site org with 6+ branch and hub networks, so feel free to swap in a different network/device name and explore.

🧪 These prompts are a starting point, not a script. You're encouraged to tweak them, combine them, or write your own entirely — ask follow-up questions, dig into something that looks interesting, or try a network/device the prompts don't mention. That experimentation is part of the lab, not a detour from it.

⏱️ **Short on time?** Labs 1-2 are marked CORE — do these first, they're the ones everyone should get to. Labs 3-6 are STRETCH goals — great if you have time, but nothing later in this module or any other module depends on them, so don't worry if class ends before you get there.

LAB 1 Meet the Network — CORE

Before you can tell this customer's story, see what you're working with.

```
Give me a health snapshot of my Meraki organization — how many
networks, devices online vs offline, and any active alerts I should
know about.
```

```
Draw me a physical diagram of my HUB11-Amsterdam network — show every
device, how they connect, and what model they are.
```

```
Summarize the IP schema for my organization — networks, VLANs, subnet
ranges, and the default gateway for each subnet.
```

```
Check all switch ports on my MS130-24X — which are active, and are any
showing errors, drops, or a duplex mismatch?
```

LAB 2 The Trouble Ticket

CORE

Users have been complaining about flaky Wi-Fi this past week. Investigate it with real historical data — not guesses.

```
Over the last 7 days on BR03-Munich, pull wireless connection stats
and failed-connection history. Identify the worst day and time window,
and which band, SSID, or access point shows the most failures.

If the data actually looks clean with nothing notable, say so
explicitly — don't manufacture an incident that isn't there.
```

```
For the worst window you just found, pull client latency history for a
few affected clients on that access point/band. Does latency spike
too, or is it purely a connection/auth problem? What does that
distinction suggest about root cause (RF interference, authentication,
capacity, DHCP, etc.)?
```

```
Pull client count history for BR03-Munich over the last 7 days — was
the network simply over capacity during the worst window, or were
there plenty of free client slots? Rule a capacity issue in or out.
```

```
Summarize this for a Network Manager who doesn't look at a dashboard
every day: what happened, when, roughly how many users it affected,
and your best-supported root cause theory. This is the trouble ticket
writeup.
```

LAB 3 The Risk Review

STRETCH

Now that the Munich fire is out, prove the rest of this customer's deployment is secure enough to trust with their business.

```
Show all firewall rules on my MX250 and flag any that allow
unrestricted inbound traffic from 0.0.0.0/0. Rank by risk and suggest
which to tighten.
```

```
Conduct a full security audit of my Meraki organization: risky
firewall rules, SSID auth modes, and rogue APs (last 7 days). Give
each finding a High/Medium/Low risk and a remediation for the top 3.
```

💡 On the last prompt: if your org has no VPN peers or firmware CVE data configured, a good answer says so explicitly instead of inventing findings — that's Claude staying grounded in real tool output rather than guessing.

LAB 4 Proving It's Fixed

STRETCH

Munich had the incident — this hub hasn't. Show the customer what "healthy" looks like here, as the contrast to the fix you just made.

```
Draw a performance dashboard for my network — client count per AP, top
bandwidth consumers, WAN uplink health, and any high-error devices.
```

```
Show the top 10 bandwidth consumers this week, broken down by device
and application. Flag anyone over 10GB.
```

```
Pull channel utilization for all my APs — are any congested on 2.4GHz
or 5GHz? Identify co-channel interference and suggest changes.
```

LAB 5 The Handoff

STRETCH

Leave this customer with clean documentation, not tribal knowledge.

```
Show the VLAN configuration on all switch ports in HUB11-Amsterdam and
flag anything misconfigured — wrong access VLANs, untagged where
tagged should be, inconsistencies. Suggest tidy-ups.
```

```
Is 802.11r fast roaming enabled on my APs? If not, what are the steps
and caveats to turn it on?
```

```
Generate a configuration audit of my HUB11-Amsterdam network — devices
& serials, SSIDs & security, firewall rules, and firmware status.
```

```
Format it as a report I could hand to a customer. If any section isn't
available for this network, say so rather than guessing.
```

💡 The Meraki demo org used in this lab doesn't have routed VLANs or a WAN VPN configured on every network, so questions about them may come back "not configured" — that's expected, and a good sign the AI isn't fabricating an answer.

LAB 6 ★ **The Story** STRETCH

Bring it all together. One narrative spanning both sites, built entirely from this chat — no dashboard, no manual copy-paste between tools.

```
Pull together everything from this session on this customer's multi-
site deployment: the trouble ticket from their Munich branch (Lab 2 —
what broke and why), plus the risk review, performance proof, and
clean handoff documentation from their Amsterdam hub. Write one
cohesive executive narrative (under 400 words) that tells the story
across both sites: what we found and fixed at the branch, what we
proactively hardened and documented at the hub, and why that
combination matters for their business as they scale to more sites.
Tone: confident, data-backed, written for a Network Manager / IT
Director deciding whether to expand this engagement.
```

PUSH IT FURTHER

```
Turn that narrative into 5 slide talking points I could present live,
each with a one-line "so what" for the business.
```

```
What's the one number or finding from this whole session that would
make the strongest case for expanding this engagement? Make the case
in two sentences.
```

🏆 Notice what you just did: reviewed a multi-site deployment, investigated a real incident at one branch, proved a fix, audited and documented a second site, and wrote an executive narrative tying both together — all without opening the Meraki dashboard once.

MODULE 3 · OPTIONAL / BONUS

↑ back to top

Splunk Optional

Two parts: the native Splunk AI Assistant, then the Splunk MCP Server connected into this app.

⏱️ **Time estimate: ~50–65 minutes total** if doing everything — Part A (AI Assistant, 3 exercises) is ~20–25 min; Part B (MCP token setup + PCI investigation) is ~25–30 min; the multi-cloud spend exercise is a further ~10–15 min bonus on top if your instance has that dataset. Each part stands alone, so feel free to split across breaks or skip Part A if short on time — it doesn't feed into Part B.

Part A — Splunk AI Assistant

This part uses Splunk's own built-in AI Assistant (native to Splunk, not this app or MCP) to write, explain, optimize, run and explain SPL — no separate setup required beyond logging in.

1. **Log in to Splunk**
   Log in with the **admin** credentials from your Splunk Cloud workshop details page.

2. **Open the Splunk AI Assistant app**
   Navigate to the **Splunk AI Assistant** app.

3. **Turn on Agent Mode**
   You'll be prompted whether to turn on Agent Mode — click **Yes**.

🎭 **Scenario:** you're a new analyst in a SOC team that uses Splunk. Despite being unfamiliar with the platform, you've heard the Splunk AI Assistant could be a good way to get value out of Splunk faster.

LAB 1 **Identify some potentially suspicious emails**

There have been reports of a high volume of emails from gmail accounts — investigate. Click **+ New Chat** first. You may be asked to approve SPL execution — click **Always execute for this session**. Results are non-deterministic, so yours may differ from this guide.

```
Is there any data in this environment that contains email logs? Use
index=main.
```

```
From those email log sources can you identify any data that contains
senders with a gmail domain?
```

```
Can you show me who the recipients of those emails are?
```

```
Can you show me the volume of emails over time from that sender to
that recipient, broken down day-by-day?
```

```
Can you summarize the findings from this chat for me in 200 words or
less?
```

LAB 2 **Support the Detection Engineering team**

Your manager wants you to get familiar with the detection engineering team's work. Click **+ New Chat**, then paste this complex detection rule and ask the assistant to explain it.

```
| tstats `security_content_summariesonly` count min(_time) as firstTime
max(_time) as lastTime
from datamodel=Web where
Web.http_user_agent =*
NOT Web.http_user_agent IN (
    "-",
    "unknown"
)
by Web.http_user_agent Web.dest Web.src Web.status
| `security_content_ctime(firstTime)`
| `security_content_ctime(lastTime)`
| `drop_dm_object_name(Web)`
| eval http_user_agent = lower(http_user_agent)
| lookup scripting_tools_user_agents tool_user_agent AS http_user_agent
OUTPUT tool
| where isnotnull(tool)
| stats count
    min(firstTime) as first_seen
    max(lastTime) as last_seen
    values(tool) as tool
by http_user_agent dest src status
| `security_content_ctime(first_seen)`
```

```
| `security_content_ctime(last_seen)`
| `http_scripting_tool_user_agent_filter`
```

```
Help describe what this SPL is doing: [paste the SPL above]
```

```
Help me understand where this detection rule came from — is it from
Splunk's ESCU content?
```

```
Help me understand what macros are, how to use them, and how Splunk
stores them.
```

```
Summarize the findings of this chat so I can document this SPL search
in my knowledge base.
```

LAB 3 **Help a friend**

An SRE who doesn't use Splunk often has handed the detection engineering team this search. First, notice it fails when you run it directly from the search bar (permissions) — then ask the AI Assistant to explain it instead. Click **+ New Chat**, then write "Help explain this SPL:" followed by the query below.

```
index=_internal (host="*") (sourcetype=aws:s3:log OR
sourcetype=aws:logs:log OR sourcetype=aws:sqsbaseds3:log OR
sourcetype=aws:metadata*:log OR sourcetype=aws:cloudwatch:log OR
sourcetype=aws:cloudwatchlogs:log OR sourcetype=aws:kinesis:log)
(datainput="*") level=ERROR
| fillnull value="" ErrorCode, ErrorDetail
| eval ErrorDetail = if((ErrorDetail == "" or ErrorDetail == "'") and
!isnull(message), message, ErrorDetail)
| table count by _time, message, host, sourcetype, datainput, ErrorCode,
ErrorDetail
| eval error_count=1
| lookup aws_health_error_type_lookup ErrorCode, ErrorDetail, sourcetype
OUTPUT ErrorType | rename error_count as count
| lookup aws_log_sourcetype_modinput_lookup sourcetype OUTPUT modinput
| timechart sum(count) by modinput
```

Does the explanation catch anything unusual about the search?

Part B — Splunk MCP Server

Exercise 1 — Create an MCP token & connect it in this app

You'll first create an encrypted access token inside Splunk itself, then paste that token into this app's Credentials page so the AI chat can use it.

1. **Log in to Splunk**
   Verify you're still logged in as **admin**.

2. **Open the Splunk MCP Server app**
   From the Splunk Apps menu, click **Splunk MCP Server**.

3. **Create an MCP Encrypted Token**
   Click **Create MCP Encrypted Token**. Set **User** to the account you logged in with (e.g. `admin` — not a real-world best practice, but fine for this lab). Leave **Audience** as `mcp`. Leave **Expiration** as Relative Time `+30d`. Click **Create**.

4. **Copy the token — this is your only chance**
   Click **Copy** and paste the token somewhere safe (Notepad/TextEdit). The full token is shown **only once** — if you close the dialog without copying it, you'll need to create a new one. Click **Close** when done.

5. **Open Credentials in this app**
   Go to the AI Assurance Lab, click ⚙ **Credentials** in the header, and scroll to the **Splunk** card.

6. **Enter the MCP Server URL and token**
   Enter the **MCP Server URL** your facilitator gave you (or your own tunnel URL if running locally — see below), paste the token from step 4 into **Auth Token**, and check **Skip TLS certificate verification** (Splunk Show instances use a self-signed certificate).

7. **Save, then test**
   Click **Save** *first*, then click **Test Connection**. **Important:** if you click Test Connection before Save, it will fail. Once saved and tested you'll see **Server Configured: Yes** and **Connection Status: Connected**.

8. **Back to chat**
   Click **Back to Chat**, then **New Chat** to start a fresh session with Splunk tools available.

> 🔒 **Using a Splunk "Show" training instance?** Those use a self-signed certificate, which will fail the connection test with a TLS/certificate error unless you check **Skip TLS certificate verification** on the Splunk card. Only enable this for a server you trust with a self-signed cert (like Splunk Show) — never for anything else.

🌐 **Running Splunk MCP locally instead?** Start your local Splunk MCP server, expose it with a tunnel (e.g. `ngrok http 8000`) so this cloud-hosted app can reach it, and use the public `https://…ngrok.io/mcp` URL in step 6 above. Your facilitator will give you the Splunk host/port 8089 and credentials your local server needs to reach Splunk itself — that's configured on the server side, not in this app.

Exercise 2 — Ask questions of your data using natural language

Scenario: you're a compliance/risk manager trying to understand PCI compliance posture. You don't know SPL, but your AI assistant is now connected to Splunk — let's find out what it can tell you.

LAB 4 **PCI compliance investigation**

Set the session preference, then work top-down from "what do I have" to "what's wrong" to "write it up".

```
For all answers in this chat session answer this in-line using text.
Only provide report format when asked.
```

```
What is the relevance of log data to PCI compliance, and what are some
example log sources that might be useful?
```

```
List and document all of the log data available in Splunk that is
important for PCI compliance.
```

```
I see there are some missing data sources – can you outline a plan for
getting this data into Splunk, and explain where some of the data
might come from?
```

```
Based on the log data in Splunk, are there any logs that indicate an
issue with PCI compliance? Outline the searches you used so I can
share them with my Splunk platform team.
```

```
Write a 500-word paragraph in report format explaining the findings
about PCI compliance in Splunk.
```

Exercise 3 — Support the business (bonus, needs Infosec Multi-cloud dataset)

Scenario: you're a financial analyst for the CFO, trying to understand cloud spend across the business.

LAB 5 **Multi-cloud spend analysis**

Requires the Infosec Multi-cloud dataset — check with your facilitator if this data isn't in your instance.

```
Provide all forthcoming replies in text format. Do you know what data
is available in Splunk, and is any of it relevant to understanding
cloud spend?
```

```
Examine the billing data structure and determine the total cloud spend
across all providers.
```

```
Break down the spend on a day-by-day basis for each of GCP, AWS and
Azure, detailing the accounts for each provider. Which accounts
generated the highest spend?
```

```
Break down the spend on a day-by-day basis for each type of service
and each cloud provider.
```

```
Show me the Splunk searches used to generate the results in this chat,
then write a 500-word report based on these findings for my CFO.
```

APPENDIX

Troubleshooting & glossary

Claude says it has no tools

Open ⚙ **Credentials** and check the status for that service. If it doesn't say **Connected ✓**, re-paste the token/key and click **Test Connection** again. Then start a new chat message — tool availability is refreshed per request.

ThousandEyes won't connect

Confirm your account has **API Access** permission and your org isn't opted out of AI features. Demo accounts from the proctor already satisfy both. Double-check you copied the full token with no extra spaces.

How to get your ThousandEyes OAuth Bearer Token

1. **Go to Users and Roles**
   In ThousandEyes, go to **Account Settings → Users and Roles**.
2. **Find the OAuth Bearer Token control**
   On the **Profile** tab, scroll to the bottom and click **Create** next to **OAuth Bearer Token** — *not* the Basic Auth token.
3. **Enter your 2-factor code**
   A 2-factor code is emailed to you. Paste it in when prompted.
4. **Copy the token immediately**
   Your token is shown once in a popup window. Copy it right away.

> ⚠️ **You only get to see it once.** If you close the popup without copying the token, you cannot retrieve it again — you'll have to **revoke** that token and repeat the whole process to generate a new one. Copy it to somewhere safe (Notepad/TextEdit) the first time.

Meraki won't connect

The shared org is pre-connected for everyone, so this is rare. If Claude says it has no Meraki tools, let your proctor know. If you added your own Meraki API key and Org ID on ⚙ **Credentials**, confirm the key is correct and the Org ID matches an org that key has access to, then click **Test Connection** again.

Splunk won't connect (bonus module)

Confirm the MCP Server URL is reachable from the internet (this app runs in the cloud, so `localhost` URLs won't work — you need a tunnel like ngrok, or the facilitator's shared URL). If your server enforces an auth token, make sure it's entered in ⚙ **Credentials** too. If you're running your own local server, check its own Splunk host/port/credentials configuration separately — that's set up on the server side, not in this app.

Quick glossary

- → **MCP** — Model Context Protocol; the standard that lets Claude use external tools.
- → **Connector** — how you add an MCP server to Claude (remote URL + OAuth, or a local command).
- → **Enterprise Agent** — a ThousandEyes vantage point that runs your tests.
- → **Shared org** — the Meraki organization everyone in the lab is already connected to; no API key or Org ID needed unless you choose to use your own.

AI Assurance — Partner MCP Day · Lab Guide · Cisco × Anthropic

ThousandEyes & Meraki are required modules; Splunk is an optional bonus add-on.